toXo

Приложение для управления задачами
и личными финансами

Техническая документация

Полное руководство по архитектуре, функциям и использованию

Автор: TheOverforge | github.com/TheOverforge

Стек технологий

Python 3.12 PyQt6 $\geq$ 6.7 SQLite v12
pyqtgraph $\geq$ 0.13 numpy pathlib datetime

13 марта 2026 г.

Содержание

1 Обзор приложения **2**
1.1 Ключевые возможности 2

2 Используемые технологии **3**
2.1 Python 3.12 3
2.2 PyQt6 $\geq$ 6.7 3
2.2.1 Ключевые классы PyQt6 4
2.3 SQLite (через встроенный `sqlite3`) 4
2.4 pyqtgraph $\geq$ 0.13 5
2.5 numpy 5

3 Архитектура проекта **5**
3.1 Feature Slice Design (FSD) 5
3.2 Структура директорий 5
3.3 Централизованные пути (shared/config/paths.py) 6
3.4 Архитектура MainWindow (mixin-состав) 6
3.5 Страницы правой панели (QStackedWidget) 7

4 Схема базы данных **7**
4.1 Таблица `tasks` (18 полей) 7
4.2 Таблица `categories` 8
4.3 Таблица `subtasks` 8
4.4 Финансовые таблицы 9
4.4.1 fin_accounts 9
4.4.2 fin_categories 9
4.4.3 fin_transactions 9
4.4.4 fin_budgets 10
4.4.5 fin_goals 10
4.5 История миграций схемы 10

5 Модели данных **11**
5.1 Task 11
5.2 Category 12
5.3 Финансовые модели 12

6 Сервисный слой **13**
6.1 TaskService 13
6.2 CategoryService 13
6.3 AnalyticsService 14
6.4 FinanceService 14

6.5 i18n — локализация . . . 15
7 Интерфейс пользователя . . . 15
7.1 Компоновка главного окна . . . 15
7.2 Левая панель . . . 16
7.2.1 Строка поиска . . . 16
7.2.2 Кнопки фильтра . . . 16
7.2.3 Сортировка . . . 17
7.2.4 Панель категорий (CategoryBar) . . . 17
7.2.5 Список задач . . . 17
7.2.6 Нижняя панель . . . 17
7.3 Редактор задачи (страница 1) . . . 18
7.3.1 Панель форматирования описания . . . 18
7.4 Страница аналитики (страница 2) . . . 18
7.5 Страница «Календарь» (страница 4) . . . 19
7.6 Страница «Финансы» (страница 5) . . . 19
7.6.1 Обзор (Overview) . . . 19
7.6.2 Транзакции . . . 19
7.6.3 Бюджеты . . . 19
7.6.4 Цели . . . 20
7.6.5 Счета . . . 20
7.6.6 Демо-данные (первый запуск) . . . 20
7.7 Командная палитра (Ctrl+K) . . . 20
8 Функции и возможности . . . 21
8.1 Управление задачами . . . 21
8.1.1 Создание задачи . . . 21
8.1.2 Переименование (inline) . . . 21
8.1.3 Отметить как выполненную . . . 21
8.1.4 Удаление . . . 21
8.1.5 Undo удаления . . . 21
8.1.6 Дублирование . . . 21
8.2 Приоритеты . . . 22
8.3 Напоминания и Дедлайны . . . 22
8.4 Теги . . . 22
8.5 Подзадачи . . . 22
8.6 Категории . . . 22
8.7 Архив . . . 22
8.8 Массовые операции . . . 22
8.9 Импорт . . . 22
8.10 Языко-зависимый посев мок-данных . . . 23
8.11 Аудио-уведомления . . . 23
8.12 Туториал . . . 23
8.12.1 Туториальная задача . . . 23
9 Горячие клавиши . . . 24
10 Кастомные виджеты . . . 24

10.1 TaskItemDelegate 25
10.2 _CalGridWidget 25
10.3 GlassFrame 25
10.4 CommandPalette 25
10.5 CategoryBar 26
10.6 _FontPickerPanel 26
10.7 CustomFontDialog 26
10.7.1 Загрузка шрифтов при запуске 27
10.8 TextColorBar / InlineColorBar 27
10.9 FocusSaveLineEdit / FocusSaveTextEdit 28
10.10 HistoryMixin 28

11 Темы оформления 28
11.1 Dark (тёмная тема) 28
11.2 Light (светлая тема) 28
11.3 Glass (iOS blur-glass тема) 29
11.4 Цветовая система финансов 29
11.5 Windows DWM — цвет строки заголовка 29
11.6 CSS-классы кнопок 30

12 Настройки 30

13 Тестирование 31

14 Известные особенности и ограничения 32

1 Обзор приложения

toXo --- настольное приложение для управления личными задачами и финансами, разработанное на языке Python с использованием фреймворка PyQt6. Интерфейс поддерживает три темы: iOS Dark Glass (тёмный), Light (светлый) и Glass (полупрозрачный). Акцентный цвет --- синий `#0a84ff`.

1.1 Ключевые возможности

- **Полный CRUD задач** — создание, просмотр, редактирование, удаление
- **Приоритеты** — четыре уровня: нет / низкий / средний / высокий
- **Закрепление** — прикрепить задачу к верхушке списка
- **Дедлайны** — уведомление в трее за день и в день дедлайна
- **Напоминания** — произвольное время с всплывающим уведомлением
- **Повторения** — ежедневно / еженедельно / ежемесячно
- **Теги** — произвольные метки через запятую; поиск по `#тег`
- **Подзадачи** — список чекбоксов внутри задачи
- **Категории (папки)** — цветные папки; drag-and-drop задач и папок
- **Фильтрация и поиск** — статус, категория, сегодняшний дедлайн, текст, тег
- **Сортировка** — по дате, алфавиту, «невыполненные первыми», вручную
- **Архив** — задачи автоматически архивируются после N дней выполнения
- **Быстрое добавление** — поле quick-add в верхней части списка
- **Массовые операции** — выделить несколько, выполнить / переместить / удалить
- **Аналитика** — 4 типа графиков, KPI-карточки, стрик, сравнение периодов
- **Страница Календарь** — сетка месяца с точками дедлайнов, bar-chart по дням
- **Финансы** — транзакции, бюджеты, цели, счета; 5 вкладок + 3 графика

- **Горячие клавиши** — полный набор для работы без мыши
- **Командная палитра** — Ctrl+K, поиск и запуск любой команды приложения
- **Масштабирование** — Ctrl+колесо или Ctrl+/−/0
- **Undo удаления** — Ctrl+Z восстанавливает последнюю удалённую задачу
- **Интерактивный туториал** — пошаговое обучение при первом запуске + закреплённая HTML-задача с описанием всех возможностей
- **Локализация** — русский и английский, переключение в настройках
- **Системный трей** — фоновая работа с иконкой и контекстным меню
- **Звуковые уведомления** — 6 пресетов звука для напоминаний
- **Три темы** — Dark, Light, Glass (iOS blur-glass)
- **Импорт** — CSV и JSON (через диалог настроек)
- **Пользовательские шрифты** — загрузка собственных `.ttf/.otf` через панель выбора шрифта

2 Используемые технологии

2.1 Python 3.12

Язык разработки. Используются следующие современные возможности:

- `from __future__ import annotations` — отложенная аннотация типов
- Union-синтаксис `int | None` для опциональных типов
- `dataclass` из модуля `dataclasses` для моделей данных
- `pathlib.Path` для работы с путями к файлам
- `datetime` с `timezone.utc` для хранения времени в UTC
- `calendar` для вычислений количества дней в месяце

2.2 PyQt6 $\geq$ 6.7

Qt 6-биндинг для Python. Используемые модули:

Модуль	Назначение
`PyQt6.QtWidgets`	Виджеты интерфейса: главное окно, кнопки, списки, диалоги, панели
`PyQt6.QtCore`	Ядро Qt: сигналы, таймеры, настройки, константы
`PyQt6.QtGui`	Графика: QPainter, шрифты, цвета, иконки, горячие клавиши
`QSystemTrayIcon`	Системный трей с меню и уведомлениями
`QSettings`	Сохранение пользовательских настроек в реестре / файле
`QGraphicsDropShadowEffect`	Тени под элементами интерфейса

2.2.1 Ключевые классы PyQt6

Класс	Применение
`QMainWindow`	Корневое окно приложения
`QSplitter`	Делитель левой и правой панелей (перетаскиваемый); также вертикальный в редакторе
`QStackedWidget`	Переключение между страницами правой панели (6 страниц)
`QListWidget`	Список задач с кастомным делегатом
`QStyledItemDelegate`	Кастомная отрисовка строк списка
`QScrollArea`	Горизонтальная прокрутка категорий; вертикальная — подзадачи
`QDrag / QMimeData`	Drag & Drop задач и категорий
`QShortcut`	Горячие клавиши
`QTimer`	Отложенные обновления; таймер напоминаний (каждые 60 с)
`QPainter`	Рисование сетки календаря и кастомных карточек финансов
`QToolButton`	Кнопка «⋯» контекстного меню задачи

2.3 SQLite (через встроенный sqlite3)

База данных хранится в файле:

- **Windows:** `%APPDATA%\toXo_app\tasks.sqlite`
- **Linux / macOS:** `~/.toXo_app/tasks.sqlite`

Схема поддерживает инкрементные миграции через `PRAGMA user_version`. Текущая версия: **v12**.

2.4 pyqtgraph ≥ 0.13

Библиотека для высокопроизводительных 2D-графиков на Qt. Используется в трёх местах:

- **Страница аналитики** — линейный, накопительный, гистограмма по дням недели, bar-chart
- **Страница «Календарь»** — bar-chart создано/выполнено по дням месяца
- **Страница «Финансы»** — grouped bar «доходы/расходы», area chart тренда баланса

Все графики используют `pg.PlotWidget` с `hideButtons()`. После добавления элементов вызывается `getViewBox().enableAutoRange()` — *нельзя* вызывать `setXRange`/`setYRange` после него, иначе элементы рисуются как сплошные блоки.

2.5 numpy

Используется для формирования массивов позиций баров в `pg.BarGraphItem` и расчётов в финансовых графиках.

3 Архитектура проекта

3.1 Feature Slice Design (FSD)

Проект организован по принципу Feature Slice Design — каждый слой знает только о нижних слоях:

`shared` → `entities` → `features` → `pages` → `widgets` → `app`

3.2 Структура директорий

Структура проекта (FSD)

```
todo_app/
├── main.py                        # Тонкий wrapper — QApplication + MainWindow
├── requirements.txt
├── app/                           # Уровень приложения
│   ├── bootstrap.py               # MainWindow (сборка, mixin-состав)
│   ├── styles/themes.py           # Dark / Light / Glass токены + apply_app_style()
│   └── providers/ ...             # theme_provider, i18n_provider, settings_provider
├── pages/                         # Полноэкранные страницы right_stack
│   ├── analytics/page.py          # AnalyticsPage (индекс 2)
│   ├── calendar/page.py           # CalendarPage (индекс 4)
│   └── finance/page.py            # FinancePage (индекс 5)
├── widgets/                       # Составные UI-виджеты
│   ├── command_palette/ui.py      # CommandPalette (Ctrl+K)
│   ├── finance_sidebar/ui.py      # Боковая панель финансов
│   └── tutorial/ui.py             # TutorialOverlay
├── features/                      # Бизнес-функции (slice по действию)
```

```
│   ├── task/
│   │   ├── filter_tasks/ui.py    # FilterMixin
│   │   ├── edit_task/ui.py       # EditorMixin
│   │   ├── manage_tasks/ui.py    # TasksMixin
│   │   └── undo_redo/ui.py       # HistoryMixin
│   ├── category/manage_categories/ui.py  # CategoriesMixin
│   ├── finance/ ...              # Финансовые features
│   ├── settings/manage_settings/ui.py    # SettingsMixin
│   └── navigation/system/ui.py   # SystemMixin (трей, уведомления, toast)
├── entities/                     # Доменные модели и репозитории
│   ├── task/
│   │   ├── model.py              # dataclass Task
│   │   ├── repository.py         # TaskDbMixin (SQL)
│   │   ├── service.py            # TaskService
│   │   └── ui/task_list.py       # TaskItemDelegate, CategoryBar, TaskListWidget
│   ├── category/
│   │   ├── model.py / repository.py / service.py
│   ├── analytics/
│   │   ├── repository.py / service.py
│   └── finance/
│       ├── models.py             # FinAccount, FinCategory, Transaction, Budget, Goal
│       ├── repository.py         # FinanceDbMixin (+ _seed_finance_mock)
│       └── service.py            # FinanceService
├── shared/                       # Код без зависимостей от верхних слоёв
│   ├── api/db/connection.py      # Database (объединяет все DbMixin)
│   ├── i18n/__init__.py          # tr(), set_language(), current_language()
│   ├── ui/widgets.py             # FocusSaveLineEdit, FocusSaveTextEdit, …
│   ├── ui/dialogs.py             # ReminderDialog, DownwardCombo, CrispCheckBox
│   ├── config/paths.py           # PROJECT_ROOT, ASSETS_DIR, ICONS_DIR, …
│   └── assets/
│       ├── icons/                # SVG-иконки
│       ├── images/               # PNG (app_icon, logo_text, …)
│       └── sounds/               # WAV уведомлений
├── tests/
│   ├── unit/ ...
│   ├── integration/ ...
│   └── ui/ ...
└── docs/documentation.tex
```

3.3 Централизованные пути (shared/config/paths.py)

```python
from pathlib import Path

PROJECT_ROOT: Path = Path(__file__).resolve().parent.parent.parent
ASSETS_DIR:  Path = PROJECT_ROOT / "shared" / "assets"
ICONS_DIR:   Path = ASSETS_DIR / "icons"
IMAGES_DIR:  Path = ASSETS_DIR / "images"
SOUNDS_DIR:  Path = ASSETS_DIR / "sounds"
```

Все модули импортируют константы из этого файла вместо самостоятельного вычисления `Path(__file__).parent...`.

3.4 Архитектура MainWindow (mixin-состав)

`app/bootstrap.py` содержит класс `MainWindow`, собранный из 7 mixin-классов:

Mixin	Файл / Ответственность
`FilterMixin`	`features/task/filter_tasks/ui.py` — фильтрация, сортировка, поиск
`EditorMixin`	`features/task/edit_task/ui.py` — редактор задачи (правая панель)
`TasksMixin`	`features/task/manage_tasks/ui.py` — создание, удаление, дублирование
`CategoriesMixin`	`features/category/manage_categories/ui.py` — категории
`SystemMixin`	`features/navigation/system/ui.py` — трей, уведомления, масштаб
`SettingsMixin`	`features/settings/manage_settings/ui.py` — диалог настроек
`HistoryMixin`	`features/task/undo_redo/ui.py` — undo стек, toast

3.5 Страницы правой панели (QStackedWidget)

Индекс	Страница	Когда показывается
0	Пустая	Нет выбранной задачи
1	Редактор	Задача выбрана в списке
2	Аналитика	Нажата кнопка «Аналитика»
3	Массовые действия	Выделено ≥ 2 задач
4	Календарь	Нажата кнопка «Календарь»
5	Финансы	Пункт «Финансы» в бургер-меню

4 Схема базы данных

4.1 Таблица tasks (18 полей)

Поле	Тип	Описание
`id`	INTEGER PK	Автоинкрементный идентификатор
`title`	TEXT NOT NULL	Заголовок задачи
`description`	TEXT NOT NULL	Описание / заметка (по умолчанию '')
`is_done`	INTEGER	0 — активна, 1 — выполнена
`created_at`	TEXT	ISO UTC время создания
`completed_at`	TEXT	ISO UTC время выполнения (NULL если не выполнена)
`updated_at`	TEXT	ISO UTC время последнего изменения

Поле	Тип	Описание
priority	INTEGER	Приоритет: 0=нет, 1=низкий, 2=средний, 3=высокий
category_id	INTEGER	FK на categories.id (NULL — без категории)
remind_at	TEXT	ISO UTC время напоминания (NULL — нет)
remind_shown	INTEGER	0 — уведомление ещё не показано, 1 — показано
deadline_at	TEXT	ISO UTC дедлайн (NULL — нет)
deadline_notified	INTEGER	0=нет, 1=«сегодня» показано, 2=«просрочено» показано
is_pinned	INTEGER	1 = задача всегда вверху списка
recurrence	TEXT	NULL / daily / weekly / monthly
tags	TEXT	Теги через запятую: "работа,срочно"
sort_order	INTEGER	Позиция при ручной сортировке (меньше = выше)
is_archived	INTEGER	1 = задача в архиве (скрыта из основного вида)

4.2 Таблица categories

Поле	Тип	Описание
id	INTEGER PK	Автоинкрементный идентификатор
name	TEXT UNIQUE	Название категории
color	TEXT	Цвет в формате HEX (#0a84ff)
sort_order	INTEGER	Порядок отображения
created_at	TEXT	ISO UTC время создания

4.3 Таблица subtasks

Поле	Тип	Описание
id	INTEGER PK	Автоинкрементный идентификатор
task_id	INTEGER NOT NULL	FK на tasks.id
title	TEXT NOT NULL	Текст подзадачи
is_done	INTEGER	0 — активна, 1 — выполнена
sort_order	INTEGER	Порядок отображения

Поле	Тип	Описание
created_at	TEXT	ISO UTC время создания

4.4 Финансовые таблицы

4.4.1 fin_accounts

Поле	Тип	Описание
id	INTEGER PK	
name	TEXT	Название счёта
type	TEXT	cash / card / savings / invest
balance	REAL	Текущий баланс
currency	TEXT	Символ валюты (₽, $, € ...)
color	TEXT	HEX-цвет карточки
created_at	TEXT	ISO UTC

4.4.2 fin_categories

Поле	Тип	Описание
id	INTEGER PK	
name	TEXT	Название (напр. «Еда», «Зарплата»)
type	TEXT	income / expense
color	TEXT	HEX
icon	TEXT	Эмодзи-иконка
created_at	TEXT	ISO UTC

4.4.3 fin_transactions

Поле	Тип	Описание
id	INTEGER PK	
type	TEXT	income / expense / transfer
amount	REAL	Сумма
currency	TEXT	Валюта
category_id	INTEGER	FK на fin_categories.id
account_id	INTEGER	FK на fin_accounts.id
date	TEXT	Дата в формате YYYY-MM-DD

Поле	Тип	Описание
`title`	TEXT	Описание операции
`note`	TEXT	Заметка
`tags`	TEXT	Теги через запятую
`is_recurring`	INTEGER	1 = повторяющаяся
`recurring_rule`	TEXT	Правило повторения
`created_at`	TEXT	ISO UTC

4.4.4 fin_budgets

Поле	Тип	Описание
`id`	INTEGER PK	
`category_id`	INTEGER	FK на `fin_categories.id`
`limit_amount`	REAL	Лимит расходов за период
`period`	TEXT	`monthly` / `weekly`
`created_at`	TEXT	ISO UTC

4.4.5 fin_goals

Поле	Тип	Описание
`id`	INTEGER PK	
`name`	TEXT	Название цели
`target_amount`	REAL	Целевая сумма
`current_amount`	REAL	Накоплено на данный момент
`deadline`	TEXT	Дата в формате `YYYY-MM-DD`
`color`	TEXT	HEX
`currency`	TEXT	Валюта
`created_at`	TEXT	ISO UTC

4.5 История миграций схемы

Версия	Изменения
v0 → v1	`+completed_at`, `updated_at`, `priority`; создана таблица `task_events`
v1 → v2	Создана таблица `categories`; `+category_id` в `tasks`; индексы
v2 → v3	`+remind_at`, `remind_shown`; индекс `idx_tasks_remind`

Версия	Изменения
v3 → v4	+deadline_at, deadline_notified; индекс idx_tasks_deadline
v4 → v5	+is_pinned, recurrence; индекс idx_tasks_pinned
v5 → v6	Создана таблица subtasks; индекс idx_subtasks_task
v6 → v7	+tags, sort_order; обратное заполнение sort_order = id
v7 → v8	+is_archived; индекс idx_tasks_archived
v8 → v9	Созданы таблицы fin_accounts, fin_categories, fin_transactions, fin_budgets, fin_goals; посев начальных демо-данных (30 дней транзакций)
v9 → v10	Очищены старые финансовые данные; повторный посев расширенным набором (90 дней транзакций, более реалистичные суммы)
v10 → v11	Посев 90 дней истории задач через _seed_task_history() для наглядной демонстрации аналитики; добавляется туториальная задача
v11 → v12	Добавлены задачи с реальными дедлайнами для страницы Календарь (просроченные, сегодняшние, будущие) через _seed_calendar_tasks()

5 Модели данных

5.1 Task

Датакласс entities/task/model.py:

```
@dataclass
class Task:
    id: int
    title: str
    description: str
    is_done: bool
    created_at: str              # ISO UTC
    completed_at: str | None = None
    updated_at: str | None = None
    priority: int = 0            # 0=none 1=low 2=med 3=high
    category_id: int | None = None
    remind_at: str | None = None
    remind_shown: bool = False
    deadline_at: str | None = None
```

```
    deadline_notified: int = 0   # 0/1/2
    is_pinned: bool = False
    recurrence: str | None = None  # daily/weekly/monthly
    tags: str = ""                # comma-separated
    sort_order: int = 0
    is_archived: bool = False
```

5.2 Category

Датакласс `entities/category/model.py`:

```
@dataclass
class Category:
    id: int
    name: str
    color: str           # HEX, e.g. "#0a84ff"
    sort_order: int
    created_at: str
    task_count: int = 0  # вычисляетсяпризапросе
```

5.3 Финансовые модели

Датаклассы в `entities/finance/models.py`:

```
@dataclass
class FinAccount:
    id: int
    name: str
    type: str       # cash/card/savings/invest
    balance: float
    currency: str   # "RUB"/"USD"/"EUR" etc.
    color: str
    created_at: str

@dataclass
class Transaction:
    id: int
    type: str       # income/expense/transfer
    amount: float
    currency: str
    category_id: int | None
    account_id: int | None
    date: str       # YYYY-MM-DD
    title: str
    note: str = ""
    tags: str = ""
    is_recurring: bool = False
    recurring_rule: str = ""
    created_at: str = ""

```

```python
@dataclass
class Budget:
    id: int
    category_id: int
    limit_amount: float
    period: str     # monthly/weekly
    created_at: str

@dataclass
class Goal:
    id: int
    name: str
    target_amount: float
    current_amount: float
    deadline: str   # YYYY-MM-DD
    color: str
    currency: str
    created_at: str
```

6 Сервисный слой

6.1 TaskService

Файл: `entities/task/service.py`

Метод	Описание
`create_task(title, desc, cat_id)`	Создаёт задачу, возвращает `id`
`list_tasks()`	Все задачи (вкл. архивированные)
`get_task(id)`	Одна задача или `None`
`update_task(id, **fields)`	Обновляет произвольные поля
`set_done(id, is_done)`	Переключает статус; при `recurrence` создаёт клон
`delete_task(id)`	Удаляет задачу
`restore_task(snap)`	Восстанавливает задачу из словаря полей (для undo)
`duplicate_task(id)`	Копирует задачу; возвращает новый `id`
`archive_old_done(days)`	Переводит в архив выполненные задачи старше N дней
`list_subtasks(task_id)`	Подзадачи задачи
`add_subtask(task_id, title)`	Создаёт подзадачу
`set_subtask_done(sub_id, v)`	Переключает подзадачу
`delete_subtask(sub_id)`	Удаляет подзадачу

6.2 CategoryService

Файл: `entities/category/service.py`

Метод	Описание
`list_categories()`	Все категории по `sort_order`, с `task_count`
`get_category(id)`	Категория по `id` или `None`
`create_category(name, color)`	Создаёт, возвращает `id`
`update_category(id, name, color)`	Изменяет название и цвет
`delete_category(id)`	Удаляет; задачи сохраняются с `category_id = NULL`
`reorder_categories(ids)`	Переупорядочивает по переданному списку `id`
`set_task_category(task_id, cat_id)`	Назначает задаче категорию

6.3 AnalyticsService

Файл: `entities/analytics/service.py`

Метод	Описание
`kpi(days)`	Показатели за период: total_completed, pct, streak, avg_hours
`kpi_with_delta(days)`	KPI + дельта vs предыдущий период; max_streak за всё время
`completed_per_day(days)`	Выполненные задачи по дням `[(date, count)]`
`created_vs_completed(days)`	`[(date, created, completed)]`
`completed_by_weekday(days)`	По дням недели 0(пн)–6(вс)
`cumulative_completed(days)`	Накопленный итог
`status_distribution(days)`	`{done: N, active: N}`
`tasks_for_day(date_str)`	`{created: [...], completed: [...]}`
`tasks_with_deadline_on(date_str)`	Задачи с дедлайном на дату
`month_data(year, month)`	`{date: (created, completed)}`

6.4 FinanceService

Файл: `entities/finance/service.py`

Метод	Описание
`period_range(days)`	Возвращает `(date_from, date_to)`; `days=0` → текущий месяц; `days=-1` → текущий год
`transactions(days)`	Список `Transaction` за период
`kpi(days)`	`{income, expenses, balance, net, avg_per_day}`
`expenses_by_category(days)`	`[(cat_name, color, amount)]`
`income_by_category(days)`	Аналогично для доходов

Метод	Описание
balance_trend(days)	[(date, balance)] — накопленный тренд баланса
budget_status()	[(Budget, cat, spent, pct)]
accounts()	Список всех счетов
goals()	Список всех целей
add_transaction(…)	Создаёт транзакцию
add_account(…)	Создаёт счёт
add_budget(…)	Создаёт бюджет
add_goal(…)	Создаёт финансовую цель

6.5 i18n — локализация

Файл: shared/i18n/__init__.py

```
from shared.i18n import tr, set_language, current_language

tr("filter.all")            # -> Все"" / "All"
tr("fin.income")            # -> Доходы"" / "Income"
tr("bulk.selected",
   count=3, noun="задачи") # именованныеаргументы
set_language("en")          # переключитьязык
current_language()          # -> "ru" | "en"
```

Язык хранится в QSettings под ключом "language". Загружается в main.py до создания MainWindow. Ключи финансового раздела используют префикс fin.* (76 ключей).

7 Интерфейс пользователя

7.1 Компоновка главного окна

Схема главного окна

Верхняя панель

Поле поиска + кнопка добавления

Фильтры: *Все / Активные / Выполненные / Сегодня / Архив*

Сортировка задач

Категории (чипсы)

Quick-add (быстрое добавление задачи)

Центральная область

Список задач	**Правая панель (right_stack)**
☐ Задача 1 ☐ Задача 2 ✓ Задача 3 ⋮	— пусто / редактор — аналитика / календарь — финансы / bulk actions

Нижняя панель

Действия: *Удалить, Аналитика, Календарь, ▯ Меню, Настройки*

Счётчики: *Показано N, Выполнено N*

7.2 Левая панель

7.2.1 Строка поиска

Поле **SearchBox**. Фильтрует задачи в реальном времени по `title` и `description`. Префикс **#тег** фильтрует по полю `tags`. Горячая клавиша: **Ctrl+F**.

7.2.2 Кнопки фильтра

Кнопка	**Поведение**
Все	Все задачи (активные + выполненные, не архив)
Активные	`is_done = 0` и `is_archived = 0`
Выполненные	`is_done = 1` и `is_archived = 0`
Сегодня	Задачи с дедлайном сегодня или просроченные
Архив	`is_archived = 1`

7.2.3 Сортировка

Режим	Описание
Новые	По `created_at` убывая
Старые	По `created_at` возрастая
А → Я	Алфавит по заголовку
Невыпол. сверху	Активные (новые), затем выполненные
Вручную	Drag-and-drop внутри списка; сохраняется в `sort_order`

Закреплённые задачи (`is_pinned = 1`) всегда отображаются выше остальных.

7.2.4 Панель категорий (CategoryBar)

Горизонтальная прокручиваемая панель. Содержит чип «Все задачи», цветные чипы категорий и кнопку «+». Поддерживает drag-and-drop задач и чипов.

7.2.5 Список задач

`TaskListWidget` с делегатом `TaskItemDelegate`. Каждая строка:

- Цветная точка категории (6×6 px)
- Круглый чекбокс (22×22 px)
- Значок приоритета (синий/жёлтый/красный)
- Значок закрепления
- Заголовок (зачёркивание для выполненных)
- Дедлайн DD.MM или дата создания
- Счётчик подзадач (X/N) если есть подзадачи

7.2.6 Нижняя панель

Элемент	Описание
«Удалить»	Удаляет выбранные задачи
«Аналитика»	Переключает на страницу аналитики (индекс 2)
«Календарь»	Переключает на страницу календаря (индекс 4)
Бургер-меню	Открывает меню: Финансы, …
«Настройки»	Открывает плавающий диалог настроек
Счётчик	«Показано: N | Выполнено: N»

7.3 Редактор задачи (страница 1)

- **EditorTitle** — поле 28 px bold; сохранение при потере фокуса или Enter
- **«⋯»** — контекстное меню задачи (дублировать, экспорт, командная палитра)
- **Pill-кнопки приоритета** — нет / низкий / средний / высокий
- **Закрепить** и выпадающий список повторений
- **Напомнить / Дедлайн** — открывает `ReminderDialog`
- **Теги** — поле через запятую
- **EditorDesc** — многострочный текст, auto-save при потере фокуса
- **Подзадачи** — раздел ниже (Enter — добавить, ▯ — удалить)

7.3.1 Панель форматирования описания

Над полем `EditorDesc` расположена строка инструментов форматирования:

- **Кнопка шрифта** (`DescFontCombo`) — показывает текущий шрифт; клик открывает/закрывает `_FontPickerPanel`
- **Кнопки размера** (`DescSizeBtn`) — уменьшить / увеличить кегль на 1 pt
- **«A ▾»** в `TextColorBar` — дополнительная кнопка открытия `_FontPickerPanel`
- **InlineColorBar** — панель цветов текста и выделения, расположена строго по центру строки (равные виджеты-распорки по краям)

Все всплывающие панели (`_FontPickerPanel`, `TextColorBar`, `InlineColorBar`) стилизованы под тёмный полупрозрачный фон (`rgba(10,14,24,0.82)`) с мягкой рамкой в тёмной/Glass-теме и светлый в Light-теме.

7.4 Страница аналитики (страница 2)

Переключатель периода: **7 / 14 / 30 дней** и произвольный диапазон.

KPI-карточки: Выполнено (±дельта), % выполнения, Стрик текущий, Макс. стрик, Среднее время.

Графики:

- Выполнено по дням — линейный
- Создано vs. выполнено — grouped bar
- Накопленный итог — area chart
- По дням недели — гистограмма пн–вс

7.5 Страница «Календарь» (страница 4)

Горизонтальный сплиттер (380 px / 620 px):

Левая часть — сетка месяца через `QPainter`: кнопки ‹/›, ячейки дней, зелёная точка = дедлайн, синяя заливка = выбранный день.

Правая часть — список задач с дедлайном на выбранный день + кнопка «+ Добавить задачу» + bar-chart по дням текущего месяца.

Поддержка drag-and-drop из списка задач на ячейку сетки — устанавливает дедлайн 09:00 UTC.

7.6 Страница «Финансы» (страница 5)

Как открыть

Нажать ☰ (бургер-меню) → «Финансы». Повторный клик закрывает.

Страница содержит 5 вкладок:

7.6.1 Обзор (Overview)

- KPI-карточки: Баланс (синий), Доходы (зелёный), Расходы (красный), Чистый доход, Ср./день
- График «Доходы vs Расходы» — grouped bar по дням/неделям (`pg.BarGraphIte`
- Два пончика QPainter — расходы по категориям / доходы по категориям
- График «Тренд баланса» — area chart (`pg.PlotDataItem` с `fillLevel=0`)
- Последние 8 транзакций

Переключатель периода: **7д / 30д / 90д / Месяц / Год**.

Вторичный цвет текста возвращается функцией `_ts()` — она возвращает hex-строку, зависящую от темы: светлая тема — из `_THEMES["light"]["text_s` Glass — `#bdd0ec`, тёмная — `#c0c0c8`. Использование hex (а не CSS rgba) обязательно для совместимости с pyqtgraph (`pg.setConfigOptions`) и `QColor(str)`.

7.6.2 Транзакции

Фильтр по типу (Все / Доходы / Расходы / Переводы) + поиск. Прокручиваемый список строк транзакций.

7.6.3 Бюджеты

Двухколоночная сетка карточек бюджета. Каждая карточка: категория, лимит, потрачено, прогресс-бар (красный при перерасходе), статус-бейдж. Кнопка «+ Добавить бюджет».

7.6.4 Цели

Сетка карточек цели с круговым прогрессом (QPainter arc). Отображает: название, накоплено / цель, дедлайн, процент.

7.6.5 Счета

Двухколоночная сетка карточек счёта с цветной полосой-акцентом. Заголовок с суммарным балансом. Кнопка «+ Добавить счёт».

7.6.6 Демо-данные (первый запуск)

При первом запуске (или при нажатии кнопки «Загрузить тестовые данные» в настройках) вызывается `_seed_finance_mock()`. Данные генерируются на текущем языке приложения (`current_language()`):

Русский (RU):

- 3 счёта: Наличные (₽12 000), Основная карта (₽98 560), Накопления (₽14 000)
- 10 категорий: 7 расходных + 3 доходных
- 90 дней транзакций (зарплата, подписки, еда, транспорт, фриланс…)
- 3 бюджета: Еда ₽15 000, Транспорт ₽5 000, Развлечения ₽8 000
- 4 цели: «Новый монитор», «Отпуск», «Автомобиль», «Финансовая подушка»

Английский (EN):

- 3 счёта: Cash ($130), Main Card ($1 090), Savings ($155)
- 10 категорий с английскими названиями (Food, Transport, Subscriptions…)
- 90 дней транзакций в долларах (суммы ₽ ÷ 90)
- 3 бюджета: Food / Transport / Entertainment
- 4 цели: «New MacBook», «Trip to Asia», «Car», «Emergency fund»

7.7 Командная палитра (Ctrl+K)

`widgets/command_palette/ui.py`. Диалог без рамки с полем поиска и списком команд (26 команд):

- Задачи: создать, удалить, дублировать, отметить выполненной
- Фильтры и сортировки
- Переключить аналитику / календарь / финансы / настройки
- Сменить язык, тему, зум

- Экспорт / импорт, массовые операции

Избранные команды (□) хранятся в `QSettings "command_palette/favorites"` как JSON-список и отображаются первыми. Правый клик в `FocusSaveTextEdit` (описание задачи) также открывает палитру.

8 Функции и возможности

8.1 Управление задачами

8.1.1 Создание задачи

Способы создать задачу

- **Ctrl+N** или кнопка **«+»** рядом с поиском
- **Поле QuickAdd** — ввести заголовок, нажать Enter
- **Страница «Календарь»** — «+ Добавить задачу» (задача с дедлайном на выбранный день)

8.1.2 Переименование (inline)

F2 или **двойной клик** по задаче. Enter / клик в стороне — сохранить; Esc — отменить.

8.1.3 Отметить как выполненную

Клик по чекбоксу, **Ctrl+D**, ПКМ → «Выполнить». При `recurrence` автоматически создаётся новый клон со сдвинутым дедлайном.

8.1.4 Удаление

Клавиша **Delete**, кнопка «Удалить» или ПКМ → «Удалить». По умолчанию — диалог подтверждения.

8.1.5 Undo удаления

Ctrl+Z — восстанавливает задачу с помощью `svc.restore_task(snap)`. Снимок хранится в стеке `HistoryMixin` (до 50 записей).

8.1.6 Дублирование

Ctrl+Shift+D или ПКМ → «Дублировать».

8.2 Приоритеты

Четыре уровня. Цветной маркер в строке задачи:

- **Нет** — без маркера
- **Низкий** — синий
- **Средний** — жёлтый
- **Высокий** — красный

8.3 Напоминания и Дедлайны

Таймер проверяет каждые 60 с. Уведомления через системный трей + звук. Дедлайны: два уровня (за день / в день + просрочено). Задачи с дедлайном сегодня попадают в фильтр «Сегодня».

8.4 Теги

Поле через запятую. Поиск **#тег** в SearchBox фильтрует задачи по тегу.

8.5 Подзадачи

Enter — добавить, чекбокс — отметить, □ — удалить. Счётчик (X/N) в основном списке.

8.6 Категории

Создание — кнопка «+»; 8 цветов. Назначение — drag-and-drop или ПКМ → «Переместить в…». Перестановка — перетащить чип на другой.

8.7 Архив

Автоматически — задачи выполненные $>$ N дней → архив при запуске. N и включение настраиваются в диалоге настроек.

8.8 Массовые операции

При выделении ≥ 2 задач (Ctrl+A или Ctrl+клик) правая панель переключается на страницу «Массовые действия» (индекс 3): выполнить / отменить, переместить в категорию, удалить.

8.9 Импорт

В диалоге настроек — «Импорт CSV» / «Импорт JSON». CSV должен содержать хотя бы колонку `title`; JSON — массив объектов.

8.10 Языко-зависимый посев мок-данных

При нажатии кнопки «Загрузить тестовые данные» (настройки → Опасная зона) вызываются два метода:

- `db._seed_task_history()` — создаёт 90 дней истории задач по категориям. Читает `current_language()` и генерирует задачи с названиями на RU или EN. В конце вызывает `_seed_tutorial_task()`.
- `db._seed_finance_mock()` — создаёт счета, категории, 90 дней транзакций, бюджеты и цели. Также языко-зависим: на EN — долларовые суммы и английские названия.

Оба метода вызываются и через миграции (v11 и v9→v10 соответственно), поэтому существующая БД тоже получает данные при обновлении.

8.11 Аудио-уведомления

Шесть пресетов в `shared/assets/sounds/`:

- `toXo_default.wav`, Bright Minimal, Deep Luxe, Double Check, Ultra Soft, Warm Tap Chime

Файл выбирается в настройках; путь формируется через `SOUNDS_DIR` из `shared/config/paths.py`.

8.12 Туториал

Запускается при первом запуске (`QSettings "tutorial_shown" = False`). `TutorialOverlay` (`widgets/tutorial/ui.py`) — полупрозрачный слой с подсветкой виджета и кнопками «Назад» / «Далее» / «Пропустить». 9 шагов.

8.12.1 Туториальная задача

При первом запуске, если задач нет, автоматически создаётся закреплённая задача-туториал (`_seed_tutorial_task()` в `shared/api/db/connection`

- Определяет текущий язык через `current_language()` и создаёт содержимое на RU или EN
- Богатое HTML-описание с цветными секциями (клавиши, фильтры, финансы, аналитика и т.д.) в `QTextEdit`-совместимом формате с inline CSS
- Задача закреплена (`is_pinned=1`), тег `"tutorial"`, приоритет 0
- Флаг первого запуска: `QSettings "tutorial_task_seeded"` — после создания задачи устанавливается в `True`

Туториальная задача также включается при нажатии кнопки «Загрузить тестовые данные» в настройках (вызывается в конце `_seed_task_history()`

9 Горячие клавиши

Комбинация	Действие
Ctrl+N	Создать новую задачу
Ctrl+F	Переместить фокус в строку поиска
Ctrl+S	Сохранить текущую задачу (принудительно)
Ctrl+D	Переключить выполнение выбранной задачи
Ctrl+Shift+D	Дублировать выбранную задачу
Ctrl+Z	Отменить последнее удаление (восстановить задачу)
Ctrl+Y	Повторить отменённое действие
Ctrl+K	Открыть командную палитру
Ctrl+Up	Перейти к предыдущей задаче
Ctrl+Down	Перейти к следующей задаче
Ctrl+A	Выделить все задачи
F2	Переименовать задачу (inline)
F11	Полноэкранный режим
Delete	Удалить выбранную задачу (если фокус не в тексте)
Escape	Назад в календарь / закрыть страницу / убрать выделение
Ctrl + «+» / «=»	Увеличить масштаб
Ctrl + «-»	Уменьшить масштаб
Ctrl + 0	Сбросить масштаб к 100%
Ctrl + колесо мыши	Увеличить / уменьшить масштаб

Поведение Escape

1. Если кнопка «←» (назад в календарь) **видима** → открыть страницу «Календарь»
2. Иначе если открыта **аналитика** (страница 2) → закрыть
3. Иначе если открыт **календарь** (страница 4) → закрыть
4. Иначе если открыты **финансы** (страница 5) → закрыть
5. Иначе → сохранить задачу, снять выделение, перейти на пустую страницу

10 Кастомные виджеты

10.1 TaskItemDelegate

`entities/task/ui/task_list.py`. Кастомный делегат `QListWidget`:

- `paint()` — цветная точка, чекбокс, приоритет, □, заголовок, счётчик, дата
- `editorEvent()` — перехватывает клики по зоне чекбокса
- `createEditor()` — `QLineEdit` для inline-редактирования
- `sizeHint()` — высота строки 44 px

Важно: PyQt6 CheckState

`index.data(Qt.ItemDataRole.CheckStateRole)` возвращает `int` (0, 1, 2), а не `Qt.CheckState`. Правильное сравнение: `val == 2`, а не `val == Qt.CheckState.Checked`. Для `setData` используется `Qt.CheckState.Checked`.

Важно: QMouseEvent в PyQt6

Используйте `event.position()` (возвращает `QPointF`), а не устаревший `event.pos()`.

10.2 _CalGridWidget

`pages/calendar/page.py`. Рисует через `QPainter`:

- Константы ячейки: `_CELL_W=58, _CELL_H=54, _HDR_H=44, _DOW_H=22`
- Сигнал `day_selected(object)` — передаёт `datetime.date`
- Сигнал `tasks_dropped_on_day(list, object)` — drag-and-drop задач на день
- `set_deadline_days(set[str])` — обновляет зелёные точки

10.3 GlassFrame

Базовый класс карточек финансового раздела (`pages/finance/page.py`). В теме Glass рисует размытый фон через `get_glass_blur_region()`, тинт и рамку вручную через `QPainter`. В тёмной/светлой теме делегирует отрисовку к `QFrame.paintEvent()` (QSS обрабатывает фон).

Подклассы: `FinKpiCard`, `_DonutWidget`, `_TxRow`, `_AccountCard`, `_BudgetCard`, `_GoalCard`.

10.4 CommandPalette

`widgets/command_palette/ui.py`. `QDialog` с `FramelessWindowHint + WA_TranslucentBackground`. Открывается по центру `centralWidget`. Поддержка избранных команд (□) и секций-разделителей.

10.5 CategoryBar

Горизонтальная панель категорий. Сигналы:

Сигнал	Данные
`category_selected`	`int` id (−1 = «Все»)
`category_rename`	`int` id
`category_delete`	`int` id
`add_category`	(нет)
`category_reordered`	`(dragged_id, target_id)`
`task_dropped`	`([task_ids], category_id)`

10.6 _FontPickerPanel

`shared/ui/widgets.py`. Плавающее окно без рамки (`Tool | FramelessWindowHint | WA_TranslucentBackground`) для выбора шрифта описания задачи.

- Список шрифтов строится из `get_font_presets()` — встроенные + пользовательские (из `QSettings "custom_fonts"`)
- В конце списка — кнопка «□ Add custom font...» (`FontPickerAdd`), открывает `CustomFontDialog`
- **Закрытие**: клик вне панели (глобальный `eventFilter`); клик по кнопке-источнику делегируется обратно в обработчик кнопки (`source_btn` pattern)
- `show_animated(x, y, source_btn)` — принимает ссылку на кнопку-триггер, чтобы не закрывать панель при повторном нажатии
- `apply_theme(theme)` — перекрашивает фон и кнопки под текущую тему
- `refresh_fonts()` — перестраивает строки шрифтов (вызывается после добавления/удаления)

Важно: source_btn pattern

Глобальный фильтр событий закрывает панель по первому клику вне неё. Если клик приходит на кнопку-триггер, фильтр пропускает его (`return False`), чтобы обработчик кнопки сам переключил видимость. Без этого панель закрывается фильтром, затем сразу открывается обработчиком.

10.7 CustomFontDialog

`shared/ui/widgets.py`. Диалог управления пользовательскими шрифтами.

- Кнопка «**Добавить**» — `QFileDialog` на `*.ttf *.otf`; файл копируется в `shared/assets/fonts/`; путь сохраняется в `QSettings("todo_app", "todo_mvp")` под ключами `"custom_font_files"` (имена файлов) и `"custom_fonts"` (семейства)
- После успешной загрузки — `QFontDatabase.addApplicationFont()` регистрирует шрифт в сессии; имя семейства берётся из `applicationFontFami`
- При ошибке (`addApplicationFont` возвращает `−1`) — файл удаляется, показывается `QMessageBox.warning`
- Кнопка «□» рядом с каждым шрифтом — удаляет из QSettings и из каталога `fonts/`
- Сигнал `fonts_changed` — оповещает `_FontPickerPanel.refresh_fonts()`

10.7.1 Загрузка шрифтов при запуске

`load_custom_fonts()` вызывается в `main.py` до `set_language()`:

```
def load_custom_fonts() -> None:
    _CUSTOM_FONTS_DIR.mkdir(parents=True, exist_ok=True)
    for fname in QSettings("todo_app", "todo_mvp").value(
            "custom_font_files", [], type=list):
        p = _CUSTOM_FONTS_DIR / fname
        if p.exists():
            QFontDatabase.addApplicationFont(str(p))
```

10.8 TextColorBar / InlineColorBar

`shared/ui/widgets.py`. Плавающие панели цветового форматирования текста `EditorDesc`.

TextColorBar — открывается при выделении текста в `EditorDesc`: кнопки цвета текста, кнопки фона (выделения) и кнопка «A ▾» для открытия `_FontPickerPanel`.

InlineColorBar — постоянная панель в строке форматирования над `EditorDesc`; позиционируется строго по центру строки через симметричные виджеты-распорки:

```
_left_w  = QWidget()  # stretch=1, содержиткнопки  + addStretch()
inline_color_bar = InlineColorBar(...)  # stretch=0
_right_w = QWidget()  # stretch=1, пустой
bar_row.addWidget(_left_w, 1)
bar_row.addWidget(inline_color_bar, 0, AlignVCenter)
bar_row.addWidget(_right_w, 1)
```

Обе панели используют одинаковый стиль фона: `rgba(10,14,24,0.82)` в тёмной/Glass-теме, светлый в Light.

10.9 FocusSaveLineEdit / FocusSaveTextEdit

`shared/ui/widgets.py`. Расширения стандартных виджетов с `on_focus_out` и `on_enter`. `FocusSaveTextEdit` дополнительно излучает `context_menu_requested(` — перехватывает стандартное контекстное меню и открывает командную палитру.

10.10 HistoryMixin

`features/task/undo_redo/ui.py`. Стек undo до 50 записей. Снимок задачи — `t.__dict__.copy()`. Восстановление — `svc.restore_task(snap)`. Toast-уведомление `_show_toast(text)` — метка в нижней части окна на 2.2 с.

MIME-типы drag-and-drop:

- Одна задача: `application/x-task-id` (строка)
- Несколько: `application/x-task-ids` (через запятую)

11 Темы оформления

Файл тем: `app/styles/themes.py`. Модульные функции: `set_theme(name)`, `current_theme()`, `apply_app_style(app)`, `apply_button_shadows(window)`.

11.1 Dark (тёмная тема)

Переменная	Значение	Назначение
bg_dark	#0d0d0f	Фон главного окна
bg_panel	rgba(28,28,30,0.92)	Стеклянные панели
glass	rgba(255,255,255,0.07)	Фон кнопок
accent	#0a84ff	iOS blue
danger	#ff453a	Красный
text	#f5f5f7	Основной текст
text_sec	#98989d	Вторичный текст

11.2 Light (светлая тема)

Переменная	Значение	Назначение
bg_dark	#E8EFF8	Мягко-голубой фон
bg_panel	#F7F8FB	Светлые панели
accent	#3B82F6	Синий акцент
text	#1A1F2E	Тёмный текст
text_sec	#52607A	Вторичный

Переменная	Значение	Назначение
separator	#D8DFE9	Разделитель

11.3 Glass (iOS blur-glass тема)

Переменная	Значение	Назначение
bg_dark	#232c3a	Тёмно-синий фон
bg_panel	#1d2430	Панели
glass	rgba(255,255,255,0.12)	Матовое стекло
accent	#5ea2ff	Более яркий синий

Эффект blur: install_glass_bg(w) устанавливает скрытый виджет _GlassBg, который рендерит себя через QGraphicsBlurEffect(radius=20) и кэширует результат в _blur_cache. Карточки (GlassFrame) используют get_glass_blur_region() для получения blurred-фрагмента.

11.4 Цветовая система финансов

Функция _fc(key) возвращает цвет в зависимости от активной темы:

Ключ	Dark/Glass	Light
"inc"	#30d158 (зелёный)	#52d38a
"exp"	#ff453a (красный)	#ff6b6b
"bal"	#0a84ff (синий)	#5ea2ff
"goal"	#bf5af2 (фиолетовый)	#c27cff
"xfer"	#ff9f0a (оранжевый)	#ffb84d

11.5 Windows DWM — цвет строки заголовка

При тёмной и Glass-теме применяется тёмная строка заголовка:

```
# main.py
def _set_dark_titlebar(window, dark: bool = True):
    import ctypes
    dwm = ctypes.windll.dwmapi
    DWMWA_USE_IMMERSIVE_DARK_MODE = 20
    mode = ctypes.c_int(1 if dark else 0)
    dwm.DwmSetWindowAttribute(hwnd, 20, byref(mode), sizeof(mode))
    DWMWA_CAPTION_COLOR = 35
    color = ctypes.c_uint32(0x000F0D0D if dark else 0x00FBF6F3)
    dwm.DwmSetWindowAttribute(hwnd, 35, byref(color), sizeof(color))
```

11.6 CSS-классы кнопок

ObjectName	Описание
`FilterBtn`	Pill-кнопка фильтра — стекло; `:checked` = синяя заливка
`NewBtn`	Круглая кнопка «+» (36×36 px) с тенью-свечением
`DangerBtn`	Красная pill-кнопка (удаление)
`TodayBtn`	Красная pill-кнопка «Сегодня»
`ArchiveBtn`	Жёлтая/оранжевая pill-кнопка архива
`AnalyticsBtn`	Синяя pill-кнопка аналитики
`CalendarBtn`	Зелёная pill-кнопка календаря
`AccentBtn`	Синяя pill-кнопка «добавить» с белым текстом
`SettingsBtn`	Серая pill-кнопка настроек
`TaskMenuBtn`	Маленькая стеклянная кнопка «⋯» в редакторе
`LangBtn`	Кнопка выбора языка / темы в настройках
`DescFontCombo`	Кнопка текущего шрифта описания; клик → `_FontPickerPanel`
`DescSizeBtn`	Кнопки ±1 pt для кегля шрифта описания
`FontPickerAdd`	Кнопка «□ Add custom font...» в нижней части `_FontPickerPanel`
`NeutralBtn`	Нейтральная pill-кнопка (синяя обводка, без заливки) — используется в диалоге настроек (Опасная зона)
`FinKpiCard`	Карточка KPI в финансах
`FinTransactionRow`	Строка транзакции
`FinAccountCard`	Карточка счёта
`FinBudgetCard`	Карточка бюджета
`FinGoalCard`	Карточка цели
`FinBudgetBar`	Прогресс-бар бюджета (нормальный)
`FinBudgetBarDanger`	Прогресс-бар бюджета (перерасход, красный)

12 Настройки

Диалог настроек — плавающий `QDialog` (не в `right_stack`). Размер 500×720, с вертикальной прокруткой (`QScrollArea`).

Раздел	Содержимое
Язык	Кнопки «RU» / «EN»; мгновенное переключение через `retranslate()`
Тема	Кнопки «Dark» / «Light» / «Glass»; применяется немедленно
Масштаб	Слайдер 70–150%; метка с текущим значением
Подтверждение удаления	Переключатель включения диалога
Автоархив	Переключатель + поле количества дней
Звук уведомлений	Выпадающий список прес►►тов + кнопка «Воспроизвести»
Импорт	Кнопки «Импорт CSV» / «Импорт JSON»
Опасная зона	«Загрузить тестовые данные» — посев задач + финансовых мок-данных на текущем языке; «Сбросить форматирование» — убрать HTML из описаний всех задач; «Удалить все данные» — полная очистка БД с последующим созданием туториальной задачи

Все настройки хранятся в `QSettings("todo_app", "todo_mvp")`. Ключи: `"language"`, `"theme"`, `"zoom"`, `"confirm_delete"`, `"auto_archive_enabled"`, `"auto_archive_days"`, `"notification_sound"`, `"custom_fonts"` (список семейств), `"custom_font_files"` (список имён файлов в `shared/assets/fonts/`), `"tutorial_task_seeded"` (флаг первого запуска туториальной задачи).

Важно: QSettings без аргументов

Вызов `QSettings()` без аргументов использует *другой* реестровый путь, чем `QSettings("todo_app", "todo_mvp")`. Все обращения к настройкам в коде должны использовать именованный конструктор, иначе значения не будут видны между модулями.

13 Тестирование

Структура тестов в `tests/`:

- `tests/unit/` — модульные тесты сервисов и моделей
- `tests/integration/` — тесты на реальной SQLite (in-memory)
- `tests/ui/` — тесты виджетов и делегатов

14 Известные особенности и ограничения

Важные технические заметки

- **pyqtgraph auto-range**: после `enableAutoRange()` нельзя вызывать `setXRange/setYRange` — отключают авто-масштаб и графики рисуются как сплошные блоки.
- **Ленивая инициализация charts**: `_ensure_charts()` вызывается при первом `refresh()` — избегает краша pyqtgraph в скрытом `QStackedWidget`.
- **Кольцевые импорты**: `entities/__init__.py` экспортирует *только* модели (`Task`, `Category`). Сервисы импортируются напрямую (`from entities.task.service import TaskService`), иначе возникает цикл через `shared.api.db.connection`.
- **Пути к ресурсам**: всегда импортировать из `shared.config.paths` — не вычислять `Path(__file__).parent...` в каждом модуле отдельно.
- **Windows DWM**: `_set_dark_titlebar` срабатывает только после `w.showMaximized()` — вызывать после.
- **QSettings без аргументов**: `QSettings()` и `QSettings("todo_app", "todo_mvp")` пишут в *разные* разделы реестра. Все модули обязаны использовать именованный конструктор.
- **opacity на QLabel**: CSS-свойство `opacity: 0.5` не работает на `QLabel` в Qt. Для полупрозрачного текста использовать `color: rgba(r, g, b, a)`.
- **Hamburger debounce**: `QMenu.exec()` блокирует поток; после закрытия меню щелчок по кнопке обрабатывается заново. Реализован антидребезг через `_hamburger_closed_at` (timestamp) + проверка 250 мс в начале `_show_hamburger_menu`.
- **_FontPickerPanel transparency**: для корректного `alpha` окна-инструмента необходимо установить `WA_TranslucentBackground`. Скруглённые углы реализованы через `setMask(QRegion)` в `resizeEvent`.
- **Finance _ts() hex**: вспомогательная функция `_ts()` в `pages/finance/page.py` возвращает hex-строку (например `#c0c0c8`), а не CSS `rgba()`. `QColor(str)` и `pyqtgraph` принимают только hex/named-цвета.